

PulseLens — System Architecture

- Version: 1.0

Purpose: Architecture reference with full research paper methodology mapping

Principle: Every non-trivial component is grounded in a peer-reviewed method from a top-tier venue (EMNLP, ACL, NeurIPS, Google DeepMind, Stanford). Hand-rolled heuristics only where no better method exists.

Why research methods matter here

A system that scrapes the web and asks an LLM to summarize it is not hard to build. The hard problem is **trustworthiness** — making every claim traceable, every score explainable, every output verifiable by the user.

Each paper applied here solves a specific failure mode:

Failure mode	Paper that solves it
Queries miss 80% of signals	Multi-HyDE — non-equivalent query fan-out
LLM summarizes instead of extracting	RASG — schema-constrained extraction as tool use
LLM invents quotes not in source	SAFE — atomic claim verification against evidence
Single-source claims distort scores	ClaimCheck — cross-source corroboration
Claims partially supported pass through	MiniCheck — lightweight per-fact validation
All claims weighted equally	FactScore — atomic precision scoring
Narrative blends everything into vague prose	STORM — multi-perspective synthesis
Chat answers confidently from irrelevant context	Self-RAG — self-reflective retrieval
Multi-hop answers drift from evidence mid-generation	FLARE — sentence-level active retrieval
Financial sentiment misread by general LLM	FinBERT — domain-specific financial model

Table of Contents

1. [System overview](#)
 2. [Pipeline graph — full DAG](#)
 3. [Agent 1 — Query Planner](#)
 4. [Agent 2 — Web Collection Workers](#)
 5. [Agent 3 — Fact Extractors](#)
 6. [Node — SAFE Atomic Verification](#)
 7. [Agent 4 — FinBERT Scorer](#)
 8. [Node — Quality Gate](#)
 9. [Node — M4 Triangulator](#)
 10. [Agent 5 — Contradiction Writers](#)
 11. [Node — M5 Signal Scorer](#)
 12. [Agent 6 — Narrative Synthesizer](#)
 13. [Agent 7 — Watch List Builder](#)
 14. [Node — Report Assembler](#)
 15. [Chat Graph — Analyst Chat](#)
 16. [Data flow diagram](#)
 17. [Paper reference index](#)
-

1. System overview



Why LangGraph — not plain Python, not LangChain

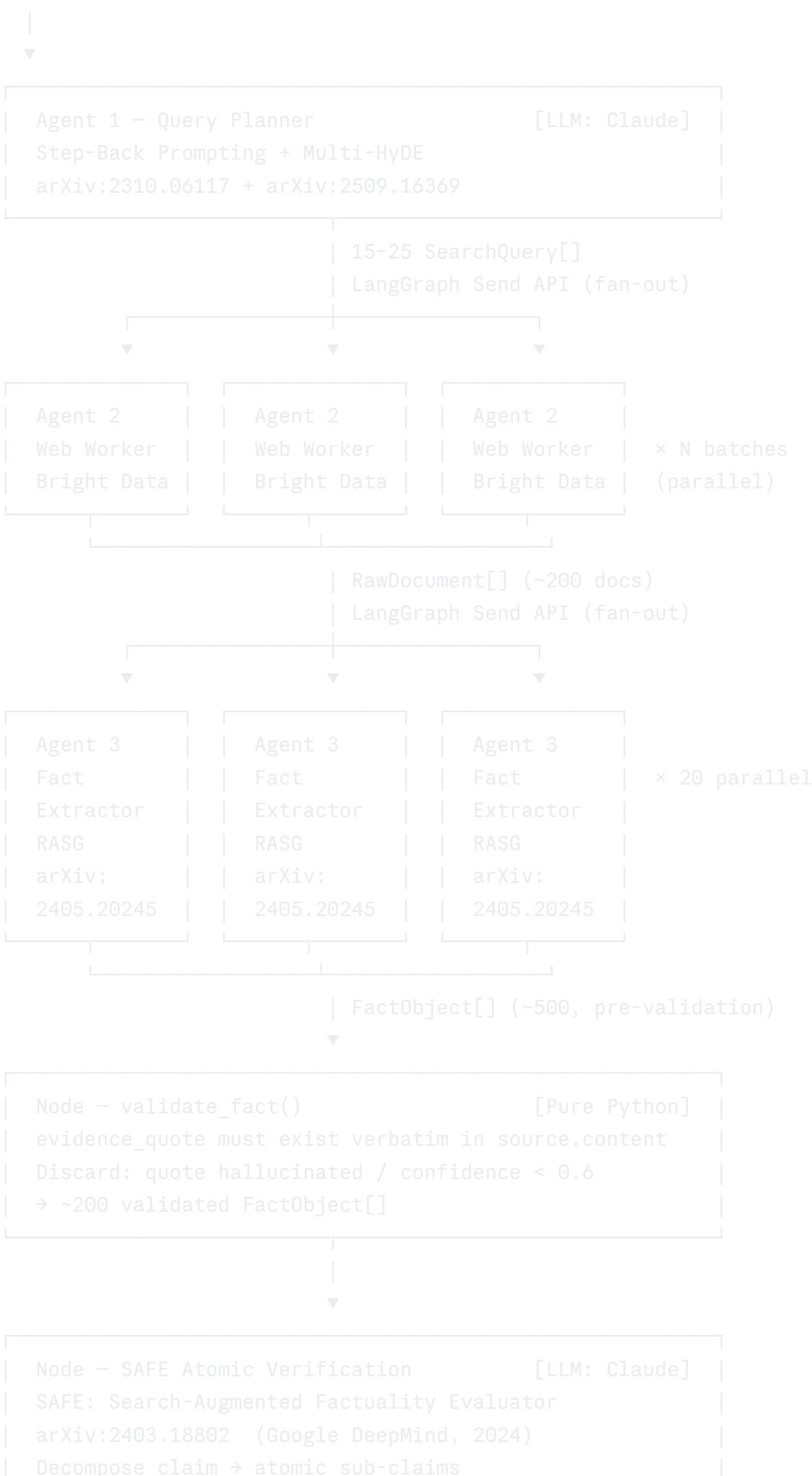
LangChain is built for linear chains ($A \rightarrow B \rightarrow C$).
PulseLens is a DAG with parallel fan-out, conditional cycles, and checkpointing.
LangChain handles this poorly. LangGraph handles it natively.

Requirement	LangGraph capability
Parallel M2/M3 fan-out (25+ URLs simultaneously)	<code>Send</code> API — true parallel node execution
Resume after failure without restarting	Built-in checkpointing to SQLite
Re-query when signal coverage is low	Conditional edges with cycle support
Persistent chat conversation history	StateGraph with thread-level state
Streaming chat tokens to frontend	First-class token streaming
Typed state shared across all nodes	<code>TypedDict</code> — enforced at every node boundary

Critical: LangGraph does not require LangChain abstractions. Every LLM call in PulseLens uses the **Anthropic SDK directly**. LangGraph is only the graph wiring — not the LLM interface.

2. Pipeline graph — full DAG

START



Verify each atomic claim against evidence_quote
Discard fact if < 50% atomic claims supported



Agent 4 – FinBERT Scorer [HuggingFace, no LLM]
FinBERT: ProsusAI/finbert
Yang et al., 2020 (HuggingFace)
Batch sentiment scoring on every fact.claim
Output: sentiment label + score (-1.0 to 1.0)



Node – Quality Gate [Pure Python]
Conditional edge: pass → proceed
fail → loop back to Agent 1 (max 2×)
Fail if: facts < 50 OR signal coverage < 4 types

expand_queries

proceed



Agent 1 again
(round 2 max)



Node – M4 Triangulator [Pure Python]
ClaimCheck ACL 2025
MiniCheck arXiv:2404.10774
FActScore arXiv:2305.14251



Agent 5
Contradiction
Writer
[LLM:Claude]

Agent 5
Contradiction
Writer
[LLM:Claude]

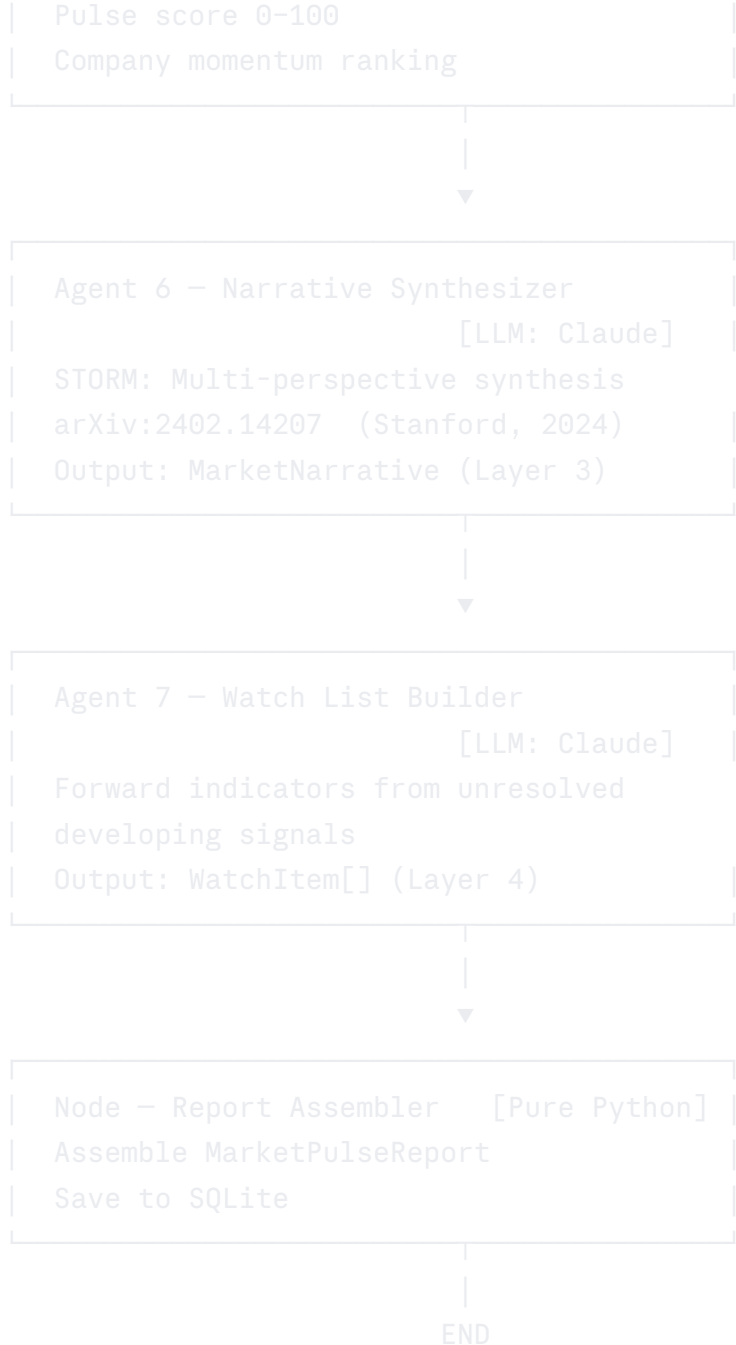
Agent 5
...
[LLM:Claude]



VerifiedClaim[]
ContradictionFlag[]



Node – M5 Signal Scorer [Pure Python]
Weighted formula: tier × recency
× factscore × corroboration



3. Agent 1 — Query Planner

File: `app/pipeline/agent1_query_planner.py`

Type: LLM — Claude (`claude-sonnet-4-20250514`)

LangGraph node: `query_planner`

Research methods applied

[PAPER 1] Step-Back Prompting

Authors: Zheng et al.
Venue: Google DeepMind, 2023
Citation: arXiv:2310.06117

What it does: Before generating queries, the LLM takes a "step back" to identify the correct level of abstraction. Instead of immediately searching for "Nvidia hiring surge", the model first asks: *"What would the evidence look like if AI infrastructure demand is accelerating?"* — then generates queries from that higher-level frame.

Why it matters here: Direct query generation from company names produces surface-level queries. Step-Back identifies the underlying signal patterns first, then generates queries that would surface evidence for or against each pattern. This prevents the common failure of fetching articles that mention the company but contain no useful signal.

Applied at: First reasoning step in the Query Planner prompt — before Multi-HyDE query generation begins.

[PAPER 2] Multi-HyDE (Multiple Hypothetical Document Embeddings)

Authors: Srinivasan et al., IIT Madras
Venue: EMNLP 2025
Citation: arXiv:2509.16369
Results: +11.2% accuracy, -15% hallucination on financial QA benchmarks

What it does: Instead of generating 1 query per topic, Multi-HyDE generates N non-equivalent hypothetical queries per signal dimension. Each query hypothesizes what a relevant document would look like if the signal were confirmed. The system then retrieves documents matching each hypothesis, dramatically increasing signal coverage.

Why it matters here: A single query "Nvidia AI hardware news" returns generic articles. Multi-HyDE generates: "Nvidia AI infrastructure job postings Q2 2025", "Nvidia data center engineering headcount expansion", "Nvidia GPU compute workforce site:linkedin.com" — each targeting a different facet of the same signal, collectively recovering evidence that no single query would find.

Applied at: Main query generation loop. Every `(company, signal_type, source_type)` triple spawns 2–3 non-equivalent queries.

Prompt design


```
python
```

```
SYSTEM_PROMPT = """
```

```
You are a financial intelligence research planner.
```

```
STEP 1 – STEP BACK (arXiv:2310.06117):
```

```
Before writing any queries, identify the abstract signals that would  
confirm or deny whether this market is accelerating.
```

```
Ask yourself: what would the evidence look like if the hypothesis were true?
```

```
STEP 2 – MULTI-HYDE DECOMPOSITION (arXiv:2509.16369):
```

```
Generate {target_count} search queries across 3 dimensions:
```

- company dimension: 1 query per company
- signal type dimension: 2-3 queries per signal type
- source type dimension: news SERP / job pages / IR pages / pricing pages

```
Rules:
```

- Non-redundant: no two queries should retrieve the same documents
- Prioritize Tier 1 (SEC, IR pages) and Tier 2 (Reuters, Bloomberg)
- Each query targets exactly ONE (company × signal_type × source_type)
- On expansion round 2: focus on low-coverage signal types: {low_signal_types}

```
Return ONLY valid JSON: List[SearchQuery]. No prose.
```

```
"""
```

Quality constraints

```
python
```

```
MIN_QUERIES = 15
```

```
MIN_SIGNAL_TYPES = 5 # must cover at least 5 of 7 signal types
```

```
MAX_EXPANSION_ROUNDS = 2 # hard stop to prevent infinite loops
```

4. Agent 2 — Web Collection Workers

File: `app/pipeline/agent2_web_workers.py`

Type: Non-LLM — async Python + Bright Data SDK

LangGraph node: `web_worker` (fanned out via `Send` API)

Research methods applied: None — pure engineering

Source tiering (assigned at collection time, never retroactively)

python

```
TIER_1_DOMAINS = {                                # weight: 1.0
    "sec.gov", "ir.nvidia.com", "ir.amd.com", "investor.intel.com",
    "investors.broadcom.com", "ir.supernmicro.com", "ir.dell.com",
    "investor.hpe.com", "investor.micron.com"
}
TIER_2_DOMAINS = {                                # weight: 0.8
    "reuters.com", "bloomberg.com", "wsj.com", "ft.com",
    "apnews.com", "businesswire.com", "prnewswire.com"
}
TIER_3_DOMAINS = {                                # weight: 0.5
    "techcrunch.com", "theverge.com", "wired.com", "arstechnica.com",
    "semianalysis.com", "tomshardware.com", "anandtech.com"
}
# Tier 4 = default for job boards, pricing pages, company blogs # weight: 0.4
```

Bright Data tool mapping

python

```
TOOL_MAPPING = {
    "serp_news":      "SERP API",          # news, general search
    "job_pages":      "Web Scraper API",    # LinkedIn, Glassdoor, Indeed
    "ir_pages":       "Web Scraper API",    # SEC EDGAR, IR pages
    "pricing_pages":  "Web Scraper API",    # pricing, distributor listings
    "dynamic_pages":  "Scraping Browser",   # JavaScript-rendered pages
    "protected":      "Web Unlocker",       # anti-bot protected sites
}
```

Parallelism and resilience

python

```
MAX_CONCURRENT_BATCHES = 10
QUERIES_PER_BATCH      = 5
RETRY_DELAYS            = [1, 2, 4]      # exponential backoff in seconds
TIMEOUT_PER_REQUEST     = 15             # seconds
CACHE_KEY               = lambda url, date: f"{url}:{date}"
```

Retry strategy: 3× with exponential backoff. On final failure: log error, skip document, continue pipeline. A failed URL never blocks the run.

Cache: keyed by (url, date) via diskcache. The same URL is never fetched twice in a single pipeline run.

5. Agent 3 — Fact Extractors

File: `app/pipeline/agent3_fact_extractors.py`

Type: LLM — Claude

LangGraph node: `fact_extractor` (fanned out via `Send` — 20 concurrent)

Research methods applied: RASG

[PAPER 3] RASG — Retrieval Augmented Structured Generation

Authors: Cesista et al.

Venue: 2024

Citation: arXiv:2405.20245

What it does: Frames information extraction as a **tool-use task**, not a text generation task. The LLM is given a strict JSON schema as a "tool definition" and must fill the schema fields. It cannot generate text outside the schema. This eliminates the primary failure mode of free-form extraction: vague, unauditable, inconsistently structured summaries that are hard to validate downstream.

Why it matters here: Without RASG: LLM writes "Nvidia seems to be expanding its AI workforce significantly" — no quote, no date, no confidence, no entity structure. With RASG: LLM fills `{entity: "Nvidia", signal_type: "hiring_momentum", claim: "Nvidia opened 23 AI infra roles this week", evidence_quote: "Nvidia has opened more than 20 positions...", confidence: 0.91}` — fully structured, auditable, downstream-ready.

Applied at: Extraction system prompt and output schema definition.

Extraction prompt

python

```
SYSTEM_PROMPT = """
```

You are a financial market intelligence extraction system.

Method: RASG – schema-constrained extraction (arXiv:2405.20245)

Extract ONLY facts EXPLICITLY STATED in the provided text.

Do NOT infer, interpret, or add information not in the text.

Return ONLY a valid JSON array. If no relevant facts, return [].

Schema for each fact object:

```
{
  "entity":          "Company name or 'market'",
  "signal_type":     "one of: hiring_momentum | product_launch | pricing_pressure
                    | strategic_messaging | investor_signal
                    | news_sentiment | supplier_risk",
  "claim":           "1 factual sentence, max 150 chars, no interpretation",
  "evidence_quote":  "exact verbatim quote from the text, max 200 chars",
  "published_date":  "ISO 8601 or null",
  "confidence":      0.0-1.0
}
```

Context:

```
  query:             {query}
  expected_signal:   {signal_type}
```

Text:

```
{document_content}
"""
```

Post-LLM validation (pure Python — most critical correctness gate)

python

```
def validate_fact(fact: FactObject, source: RawDocument) -> bool:
    """
    CRITICAL: evidence_quote must appear VERBATIM in source.content.
    If it doesn't, the LLM invented the quote – discard the entire fact.
    This is the single most important anti-hallucination gate in the system.
    """
    if fact.evidence_quote not in source.content:
        return False    # hallucinated quote
    if len(fact.claim) > 150:
        return False
    if fact.confidence < 0.6:
        return False
    if fact.entity not in KNOWN_ENTITIES:
        return False
    return True
```

Any fact failing this validation is silently discarded before it enters the rest of the pipeline. No downstream component ever sees a hallucinated fact.

6. Node — SAFE Atomic Verification

File: `app/pipeline/node_validate_and_split.py`

Type: LLM call (Claude) + pure Python logic

LangGraph node: `validate_and_split`

[PAPER 4] SAFE — Search-Augmented Factuality Evaluator

Authors: Wei et al., Google DeepMind
Venue: 2024
Citation: arXiv:2403.18802

What it does: After extracting each fact, SAFE decomposes the `claim` into atomic, independently verifiable sub-claims. It then checks whether each atomic claim is actually supported by the `evidence_quote`. Facts where fewer than 50% of atomic claims are supported are discarded.

This catches a failure mode that the `evidence_quote` presence check cannot: the quote is real, but the LLM's `claim` goes further than what the quote actually says. Example:

- Evidence quote: "Nvidia opened 20+ AI infra positions"
- Claim: "Nvidia is aggressively expanding its AI infrastructure team and expects to double headcount" ← the "double headcount" part is not in the quote

SAFE catches this by splitting the claim into atomics:

1. "Nvidia opened 20+ AI infra positions" ✓ supported
2. "Nvidia expects to double headcount" ✗ not supported → downgrade

Why it matters here: This is the strongest anti-hallucination mechanism in the system. Evidence quote presence only checks that the quote exists. SAFE checks that the claim accurately represents what the quote says.

Applied at: Between Agent 3 output and FinBERT scoring. Every validated FactObject goes through SAFE before its sentiment is scored.

Implementation

python

```
def atomic_split_and_verify(fact: FactObject, llm) -> Optional[FactObject]:
    # Step 1: decompose claim into atomic sub-claims
    atomic_prompt = f"""
    Decompose this claim into atomic, independently verifiable facts.
    Each atomic fact must be a single assertion that cannot be split further.
    Return a JSON array of strings only.

    Claim: "{fact.claim}"
    """
    atomic_claims: List[str] = llm.call(atomic_prompt, output_type="json")

    # Step 2: verify each atomic claim against evidence_quote
    supported = []
    for atomic in atomic_claims:
        support_prompt = f"""
        Is this atomic claim directly supported by the evidence quote?
        Answer ONLY "yes" or "no".

        Atomic claim:    "{atomic}"
        Evidence quote: "{fact.evidence_quote}"
        """
        answer = llm.call(support_prompt).strip().lower()
        if answer == "yes":
            supported.append(atomic)

    # Step 3: discard fact if support ratio < 50%
    support_ratio = len(supported) / max(len(atomic_claims), 1)
    if support_ratio < 0.5:
        return None    # discard

    fact.atomic_claims = supported
    fact.safe_verified = True
    return fact
```

7. Agent 4 — FinBERT Scorer

File: `app/pipeline/agent4_finbert_scorer.py`

Type: Non-LLM — HuggingFace inference

LangGraph node: `finbert_scorer`

[PAPER 5] FinBERT

Authors: Yang et al.

Venue: 2020

Source: HuggingFace – ProsusAI/finbert

What it does: BERT model fine-tuned on financial corpora: earnings call transcripts, analyst reports, financial news articles. Outputs sentiment classification (positive / negative / neutral) + confidence score per claim.

Why not use Claude for sentiment:

- FinBERT is faster: <1ms per claim on CPU vs ~500ms per LLM call
- FinBERT is free: no API cost
- FinBERT is more accurate on financial jargon: understands domain-specific phrases like "guidance raised", "margin compression", "channel inventory digestion" that general LLMs sometimes misclassify
- Deterministic: same claim always produces the same score

Applied at: Runs on every FactObject after SAFE verification. Scores are used in M4 triangulation weighted sentiment and M5 pulse scoring.

Implementation

python

```
from transformers import pipeline

finbert = pipeline(
    "sentiment-analysis",
    model="ProsusAI/finbert",
    tokenizer="ProsusAI/finbert",
    device="cpu",          # set to "cuda" if GPU available
)

def score_batch(facts: List[FactObject], batch_size: int = 32) -> List[FactObject]:
    claims = [f.claim for f in facts]
    results = finbert(claims, batch_size=batch_size, truncation=True)

    for fact, result in zip(facts, results):
        label = result["label"]      # "positive" | "negative" | "neutral"
        score = result["score"]      # 0.0-1.0 confidence from FinBERT

        fact.sentiment = label
        fact.sentiment_score = (
            score if label == "positive" else
            -score if label == "negative" else
            0.0
        )
    return facts
```

8. Node — Quality Gate

File: `app/pipeline/node_quality_gate.py`

Type: Pure Python — conditional edge

LangGraph node: `quality_gate`

Research methods applied: None

python

```
def quality_gate(state: PipelineState) -> Literal["expand_queries", "proceed"]:
    facts = state["scored_facts"]
    covered_signals = {f.signal_type for f in facts}
    rounds = state["query_expansion_rounds"]

    if rounds >= 2:
        # Hard stop – prevent infinite loop
        return "proceed"

    if len(facts) < 50 or len(covered_signals) < 4:
        # Signal coverage insufficient – identify gaps and loop back
        state["low_signal_types"] = [
            st for st in SignalType if st not in covered_signals
        ]
        state["query_expansion_rounds"] += 1
        return "expand_queries"

    return "proceed"
```

On `expand_queries`: routes back to Agent 1 with `low_signal_types` injected into state. Agent 1 generates 5-10 gap-filling queries targeting only the underrepresented signal types.

9. Node — M4 Triangulator

File: `app/pipeline/node_triangler.py`

Type: Pure Python

LangGraph node: `triangler`

[PAPER 6] ClaimCheck

Authors: Putta et al.
Venue: ACL / KnowledgeNLP Workshop 2025
Results: 62.6% verdict prediction accuracy on AVeriTeC benchmark

What it does: Two-strategy corroboration pipeline:

1. Claim-matching: checks if the claim is corroborated by established fact-check sources with known verdicts
2. Novel claim processing: generates verification questions → retrieves web evidence → assigns a verdict

Applied here (adapted): The corroboration logic: a claim is only raised as a VerifiedClaim if it is supported by ≥ 2 independent source domains. A Tier 1 source (SEC, IR page) alone is sufficient to override the corroboration requirement — these are authoritative by definition.

[PAPER 7] MiniCheck

Authors: Tang et al.
Venue: 2024
Citation: arXiv:2404.10774
Results: F1 equivalent to GPT-4, 400× faster

What it does: Lightweight fact-checking model. For each FactObject, checks whether the extracted claim is actually supported by the source document it came from. Catches cases where: the evidence_quote is present in the document, but the LLM's claim misrepresents or overstates what the quote says.

Applied here: Pre-triangulation validation step. Every FactObject is run through MiniCheck before being grouped for corroboration analysis. Facts not supported by their source are discarded.

[PAPER 8] FActScore — Fine-grained Atomic Evaluation

Authors: Min et al.
Venue: EMNLP 2023
Citation: arXiv:2305.14251

What it does: Scores the precision of a generated text by breaking it into atomic facts and checking the fraction that are independently supported. A VerifiedClaim with FActScore = 0.9 has 90% of its atomic facts supported.

Applied here: After triangulation, each VerifiedClaim is assigned a `factscore`. This is used in M5 as a quality multiplier in the pulse score formula: high factscore claims contribute more to the final score than low factscore claims. Prevents low-quality claims from inflating the score.

Triangulation logic

python

```
def triangulate(
    facts: List[FactObject],
) -> tuple[List[VerifiedClaim], List[tuple]]:
```

```
    # Step 1: MiniCheck validation (arXiv:2404.10774)
```

```
    validated = [f for f in facts if minicheck_validate(f)]
```

```
    # Step 2: Group by (entity, signal_type) – ClaimCheck grouping
```

```
    groups = defaultdict(list)
```

```
    for fact in validated:
```

```
        groups[(fact.entity, fact.signal_type)].append(fact)
```

```
    verified_claims = []
```

```
    contradiction_pairs = []
```

```
    for (entity, signal_type), group in groups.items():
```

```
        unique_domains = {extract_domain(f.source_url) for f in group}
```

```
        has_tier1 = any(f.source_tier == 1 for f in group)
```

```
    # ClaimCheck corroboration requirement (ACL 2025)
```

```
    if len(unique_domains) < 2 and not has_tier1:
```

```
        continue # insufficient corroboration – skip
```

```
    # Contradiction detection
```

```
    sentiments = {f.sentiment for f in group}
```

```
    is_contradicted = "positive" in sentiments and "negative" in sentiments
```

```
    # FactScore: atomic precision (arXiv:2305.14251)
```

```
    all_atomics = [a for f in group for a in (f.atomic_claims or [])]
```

```
    supported = [a for a in all_atomics if is_atomic_supported(a, group)]
```

```
    factscore = len(supported) / max(len(all_atomics), 1)
```

```
    # Recency-weighted sentiment (tier × recency)
```

```
    weighted_sent = calculate_weighted_sentiment(group)
```

```
    recency = calculate_recency_score(group)
```

```
    confidence = calculate_final_confidence(
```

```
        corroboration_count = len(unique_domains),
```

```
        source_tiers = {f.source_tier for f in group},
```

```
        recency_score = recency,
```

```
        factscore = factscore,
```

```
    )
```

```
    verified_claims.append(VerifiedClaim(
```

```
        entity = entity,
```

```
        signal_type = signal_type,
```

```

signal_type = signal_type,
summary = generate_summary(group),
supporting_facts = [f.fact_id for f in group],
corroboration_count = len(unique_domains),
source_tiers_present = sorted({f.source_tier for f in group}),
weighted_sentiment = weighted_sent,
recency_score = recency,
final_confidence = confidence,
factscore = factscore,
is_contradicted = is_contradicted,
contradiction_note = None, # written by Agent 5
))

if is_contradicted:
    contradiction_pairs.append((entity, signal_type, group))

return verified_claims, contradiction_pairs

def recency_weight(published_date: str, window_days: int = 7) -> float:
    age = days_since(published_date)
    return 0.0 if age > window_days else 1.0 / (age + 1)
    # Day 0 = weight 1.0 · Day 3 = 0.25 · Day 6 = 0.14

def calculate_weighted_sentiment(facts: List[FactObject]) -> float:
    tier_w = {1: 1.0, 2: 0.8, 3: 0.5, 4: 0.4}
    total = sum(tier_w[f.source_tier] * recency_weight(f.published_date)
                for f in facts)
    wmean = sum(f.sentiment_score * tier_w[f.source_tier]
                * recency_weight(f.published_date) for f in facts)
    return wmean / total if total > 0 else 0.0

```

Contradiction rule

When contradiction is detected, the system does **NOT** blend the signals into a neutral statement. Both sides are preserved explicitly as `ContradictionFlag` and surfaced in the UI as-is. Users must make their own judgment. Blending contradictory signals is a form of misinformation.

10. Agent 5 — Contradiction Writers

File: `app/pipeline/agent5_contradiction_writer.py`

Type: LLM — Claude

LangGraph node: `contradiction_writer` (parallel, one per flagged pair)

Research methods applied: None — requires LLM judgment

Why a separate agent: Contradiction *detection* is pure Python (comparing sentiment labels across domains). Writing a precise, analyst-grade note that accurately and symmetrically represents both sides requires language understanding. Separating these ensures the detection step is deterministic and the note-writing step is isolated and independently testable.

System prompt:

```
You are writing a contradiction note for a financial intelligence report.
Present BOTH sides accurately and symmetrically.
Do NOT lean toward either interpretation.
Do NOT blend the conflicting signals into a neutral statement.
Do NOT editorialize or add your own interpretation.
End with: "Recommend manual review before acting on this signal."
```

```
Entity:           {entity}
Signal type:       {signal_type}
Positive evidence: {positive_facts_json}
Negative evidence: {negative_facts_json}

Return: a single paragraph, max 100 words.
```

11. Node — M5 Signal Scorer

File: `app/pipeline/node_signal_scorer.py`

Type: Pure Python — deterministic formula

LangGraph node: `signal_scorer`

Research methods applied: None — domain-specific weighted formula

Signal weights

```
python

SIGNAL_WEIGHTS = {
    "investor_signal": 0.25,    # highest — direct financial impact
    "news_sentiment": 0.20,
    "pricing_pressure": 0.18,   # direct margin impact
    "strategic_messaging": 0.15,
    "hiring_momentum": 0.12,
    "product_launch": 0.07,
    "supplier_risk": 0.03,     # low weight but triggers status override
}
```

Scoring formula

python

```
def calculate_signal_score(
    claims: List[VerifiedClaim],
    signal_type: str,
) -> float:
    """
    signal_score incorporates:
    - weighted_sentiment (from FinBERT + tier + recency)
    - final_confidence (corroboration depth)
    - factscore (FActScore atomic precision – arXiv:2305.14251)
    - source tier (Tier 1 evidence contributes more)
    - contradiction penalty (contradicted claims weighted 50% less)
    """
    relevant = [c for c in claims if c.signal_type == signal_type]
    if not relevant:
        return 0.0

    tier_rank = {1: 4, 2: 3, 3: 2, 4: 1}

    total_w, weighted_sum = 0.0, 0.0
    for c in relevant:
        base_w = (
            c.final_confidence
            * c.factscore
            * max(tier_rank.get(t, 1) for t in c.source_tiers_present)
        )
        w = base_w * (0.5 if c.is_contradicted else 1.0)
        weighted_sum += c.weighted_sentiment * w
        total_w += w

    return weighted_sum / total_w if total_w > 0 else 0.0


def calculate_pulse_score(claims: List[VerifiedClaim]) -> tuple[float, float]:
    signal_scores = {st: calculate_signal_score(claims, st) for st in SIGNAL_WEIGHTS}
    raw = sum(signal_scores[st] * w for st, w in SIGNAL_WEIGHTS.items())
    pulse = (raw + 1) / 2 * 100 # normalize [-1,1] → [0,100]
    confidence = mean([c.final_confidence * c.factscore for c in claims])
    return round(pulse, 1), round(confidence, 2)


def classify_pulse_status(
    score: float,
    has_supplier_risk: bool,
    contradiction_rate: float,
) -> PulseStatus:
```

```
) -> PulseStatus
    if contradiction_rate > 0.4:         return PulseStatus.VOLATILE
    if has_supplier_risk and score < 55: return PulseStatus.RISK_RISING
    if score >= 70:                       return PulseStatus.HEATING_UP
    if score >= 45:                       return PulseStatus.STABLE
    return PulseStatus.COOLING_DOWN
```

12. Agent 6 — Narrative Synthesizer

File: `app/pipeline/agent6_narrative_synthesizer.py`

Type: LLM — Claude

LangGraph node: `narrative_synthesizer`

[PAPER 9] STORM — Synthesis Through Outline, Research, and Multi-perspective

```
Authors:  Shao et al., Stanford University
Venue:    2024
Citation: arXiv:2402.14207
```

What it does: STORM generates long-form, grounded intelligence by:

1. Identifying distinct perspectives that need to be addressed
2. Reasoning through each perspective independently using its evidence
3. Synthesizing where perspectives agree, where they diverge, and why

This prevents the most common failure of single-pass generation:

LLM averages all signals into vague, uncommitted prose like
"the market shows mixed signals across several dimensions."

Applied here: Each of the 4 signal layers (hiring, pricing, product, investor) is treated as a distinct perspective. Agent 6 reasons through each independently, then synthesizes the interactions — where they confirm each other, where they create tension, and what that tension means for the market outlook.

System prompt

You are a senior market analyst writing for buy-side investment teams.

Method: STORM multi-perspective synthesis (arXiv:2402.14207)

STEP 1 – ANALYZE EACH PERSPECTIVE INDEPENDENTLY:

For each signal type, reason through what the evidence shows on its own.
What does hiring data say? What does pricing data say? What does investor signal data say? What does product activity say?

STEP 2 – IDENTIFY AGREEMENTS AND TENSIONS:

Where do perspectives point in the same direction? Where do they conflict?
What is unusual compared to what you would normally expect?

STEP 3 – SYNTHESIZE:

narrative_headline: 1 analyst-grade sentence. No filler language.
narrative_body: 3-5 sentences. Explain cross-signal causality.
Each sentence must cite ≥ 1 [claim_id].
anomalies: patterns that don't fit normal expectations.

Rules:

- Never predict stock prices
- If signals conflict, state it explicitly – do not smooth it over
- Separate "what evidence shows" from "what it may mean"
- Inferences must start with "This may suggest..." or "Based on..."

Return ONLY valid JSON matching the MarketNarrative schema.

Verified claims: {verified_claims_json}
Signal scores: {signal_scores_json}
Company rankings: {company_rankings_json}

Post-generation validation

python

```
def validate_narrative(
    narrative: MarketNarrative,
    valid_claim_ids: set[str],
) -> list[str]:
    errors = []
    cited = re.findall(r'\[claim_[a-z0-9]+\]', narrative.narrative_body)
    for cid in cited:
        if cid.strip('[]') not in valid_claim_ids:
            errors.append(f"Invalid claim_id: {cid}")
    for anomaly in narrative.anomalies:
        for fid in anomaly.fact_ids:
            if fid not in valid_fact_ids:
                errors.append(f"Anomaly references non-existent fact: {fid}")
    return errors

# On validation failure: one retry with errors injected into prompt
```

13. Agent 7 — Watch List Builder

File: `app/pipeline/agent7_watch_list_builder.py`

Type: LLM — Claude

LangGraph node: `watch_list_builder`

Research methods applied: None — forward-looking synthesis

Runs after Agent 6 so it has the completed market narrative as context.

System prompt:

Based on the evidence and the market narrative above, identify 3-5 forward indicators to monitor next week.

Focus ONLY on signals that are currently developing but not yet resolved. Do not invent items not supported by evidence.

For each WatchItem:

title:	what to watch (max 10 words)
rationale:	why it matters to a buy-side analyst (2 sentences)
trigger:	the specific condition that would confirm or deny this
signals_pointing_there:	fact_ids or claim_ids pointing toward this developing
urgency:	one of: this_week next_2_weeks this_month

Return ONLY a valid JSON array of WatchItem objects.

Market narrative: {market_narrative_json}

Verified claims: {verified_claims_json}

14. Node — Report Assembler

File: `app/pipeline/node_report_assembler.py`

Type: Pure Python

LangGraph node: `report_assembler`

Research methods applied: None

Assembles all pipeline outputs into a single `MarketPulseReport` object and saves it to SQLite. LangGraph checkpoint is also saved here.

python

```
def assemble_report(state: PipelineState) -> MarketPulseReport:
    report = MarketPulseReport(
        report_id = generate_uuid(),
        market = state["market"],
        time_window = state["time_window"],
        generated_at= now_iso(),

        # Layer 1
        pulse_score = state["signal_scores"]["pulse_score"],
        pulse_status = state["signal_scores"]["pulse_status"],
        pulse_confidence = state["signal_scores"]["pulse_confidence"],
        trend_vs_previous = None, # MVP: no historical data

        # Layer 2
        top_signals = build_top_signals(state["verified_claims"]),
        company_narratives = state["company_narratives"],
        news_items = build_news_items(state["scored_facts"]),

        # Layer 3 + 4
        market_narrative = state["market_narrative"],
        contradictions = state["contradictions"],
        grounded_brief = build_grounded_brief(state),

        # Meta
        evidence_count = len(state["scored_facts"]),
        source_count = len({f.source_url for f in state["scored_facts"]}),
        signal_breakdown= state["signal_scores"]["breakdown"],
    )

    save_to_sqlite(report)
    return report
```

15. Chat Graph — Analyst Chat

File: `app/chat/graph.py` + `app/chat/agent8_analyst_chat.py`

Type: Separate LangGraph StateGraph — runs on demand per user query

Does NOT re-run the pipeline — queries from existing SQLite report

[PAPER 10] Self-RAG — Self-Reflective Retrieval Augmented Generation

Authors: Asai et al.
Venue: NeurIPS 2023
Citation: arXiv:2310.11511

What it does: LLM decides *when* to retrieve, not just *what* to retrieve. Before generating, the model emits special reflection tokens:

- `[IsREL]` — is the retrieved evidence relevant to the query?
- `[IsSUP]` — is the claim I'm about to make supported by evidence?
- `[IsUSE]` — is this response actually useful to the user?

If evidence is insufficient (`[IsSUP] = no`), the model pauses and calls `search_facts()` again with a refined query before continuing.

Why it matters here: Static RAG retrieves top-k facts once and generates regardless. Self-RAG prevents the common failure: answering confidently from retrieved context that doesn't actually support the answer.

[PAPER 11] FLARE — Forward-Looking Active REtrieval

Authors: Jiang et al.
Venue: EMNLP 2023
Citation: arXiv:2305.06983

What it does: Instead of retrieve-once-then-generate, FLARE generates the answer sentence by sentence. When the model's confidence on a sentence drops below a threshold, it pauses, formulates an implicit query from the uncertain sentence, retrieves additional evidence, and regenerates that sentence with the new context.

Why it matters here: Complex multi-hop questions like "Why is AMD gaining share and what does that mean for Nvidia's Q3?" require evidence for at least two separate claims. A single retrieval pass gets evidence for one but not both. FLARE fetches the second piece of evidence when it realizes it needs it.

Chat state

python

```
class ChatState(TypedDict):
    report_id: str
    history: List[ChatMessage] # last 5 exchanges only
    current_query: str
    retrieved_facts: List[FactObject]
    response: str
    cited_fact_ids: List[str]
    retrieval_rounds: int # for FLARE tracking
```

Chat tools

python

```
@tool
def search_facts(query: str, top_k: int = 10) -> List[FactObject]:
    """
    Semantic search over fact embeddings for this report.
    Uses sentence-transformers embeddings stored in SQLite.
    """

@tool
def get_claim(claim_id: str) -> VerifiedClaim:
    """Retrieve a specific verified claim by ID."""

@tool
def get_company_narrative(ticker: str) -> CompanyNarrative:
    """Get the full company narrative for a specific company."""
```

Node sequence

```
[retrieve_facts]
    ↓
[build_prompt]      inject evidence + last 5 history exchanges
    ↓
[Agent 8: Analyst Chat]  Self-RAG + FLARE
    ↓
[validate_citations]    all [fact_id] must exist in DB
    ↓ valid              ↓ invalid (1 retry max)
[stream to frontend]    [retry with hallucinated IDs listed in prompt]
```

Citation validation

python

```
def validate_citations(response: str, valid_ids: set[str]) -> tuple[bool, list]:
    cited = re.findall(r'\[fact_[a-z0-9]+\]', response)
    hallucinated = [fid for fid in cited if fid.strip('[]') not in valid_ids]
    return (len(hallucinated) == 0), hallucinated
```

On failure, retry prompt includes:

"The following fact IDs you cited do not exist in this report: {list}.

Revise your response using only the evidence provided."

Context overflow management

python

```
MAX_CONTEXT_TOKENS = 2000
```

```
def trim_context(history, facts):
    """
    When history + evidence exceeds limit:
    Keep the 3 most-cited facts + 3 most recent exchanges.
    Drop everything else.
    """
```

16. Data flow diagram





17. Paper reference index

Complete reference list — every paper used in PulseLens.

[1] Step-Back Prompting

Title: Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models

Authors: Huaixiu Steven Zheng et al.

Venue: Google DeepMind, 2023

Link: <https://arxiv.org/abs/2310.06117>

Applied: Agent 1 – Query Planner (first reasoning step before query generation)

[2] Multi-HyDE

Title: Enhancing Financial RAG with Agentic AI and Multi-HyDE

Authors: Srinivasan et al., IIT Madras

Venue: EMNLP 2025

Link: <https://arxiv.org/abs/2509.16369>

Results: +11.2% accuracy, -15% hallucination on financial QA

Applied: Agent 1 – Query Planner (main query decomposition loop)

[3] RASG

Title: RASG: Retrieval Augmented Structured Generation
Business Document Information Extraction As Tool Use
Authors: Cesista et al.
Venue: 2024
Link: <https://arxiv.org/abs/2405.20245>
Applied: Agent 3 – Fact Extractor (schema-constrained extraction prompt)

[4] SAFE

Title: Long-form Factuality in Large Language Models
(introduces SAFE: Search-Augmented Factuality Evaluator)
Authors: Jerry Wei et al., Google DeepMind
Venue: 2024
Link: <https://arxiv.org/abs/2403.18802>
Applied: Node – validate_and_split (atomic claim decomposition + verification)

[5] FinBERT

Title: FinBERT: Financial Sentiment Analysis with Pre-trained Language Models
Authors: Yang et al.
Venue: 2020
Source: <https://huggingface.co/ProsusAI/finbert>
Applied: Agent 4 – FinBERT Scorer (sentiment classification on every fact.claim)

[6] ClaimCheck

Title: ClaimCheck: Automated Fact-Checking via Web Evidence
Authors: Putta et al.
Venue: ACL / KnowledgeNLP Workshop 2025
Results: 62.6% verdict accuracy on AVeriTeC benchmark
Applied: Node – M4 Triangulator (cross-source corroboration logic)

[7] MiniCheck

Title: MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents
Authors: Tang et al.
Venue: 2024
Link: <https://arxiv.org/abs/2404.10774>
Results: GPT-4-level F1, 400× faster
Applied: Node – M4 Triangulator (per-fact validation before corroboration)

[8] FActScore

Title: FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation
Authors: Min et al.
Venue: EMNLP 2023
Link: <https://arxiv.org/abs/2305.14251>
Applied: Node – M4 Triangulator (atomic precision score stored as VerifiedClaim.factscore)
Node – M5 Signal Scorer (factscore used as quality multiplier in pulse formula)

[9] STORM

Title: Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models
(introduces STORM: Synthesis Through Outline, Research, Multi-perspective)
Authors: Shao et al., Stanford University
Venue: NAACL 2024
Link: <https://arxiv.org/abs/2402.14207>
Applied: Agent 6 – Narrative Synthesizer (multi-perspective market brief generation)

[10] Self-RAG

Title: Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection
Authors: Asai et al.
Venue: NeurIPS 2023
Link: <https://arxiv.org/abs/2310.11511>
Applied: Agent 8 – Analyst Chat (self-reflective retrieval decisions)

[11] FLARE

Title: Active Retrieval Augmented Generation
(introduces FLARE: Forward-Looking Active REtrieval)
Authors: Jiang et al.
Venue: EMNLP 2023
Link: <https://arxiv.org/abs/2305.06983>
Applied: Agent 8 – Analyst Chat (sentence-level active retrieval for multi-hop questions)

*Version 1.0 — Architecture reference with full paper methodology mapping.
Every non-trivial design decision traces back to a peer-reviewed method.*